Fine-Grained Selective Recomputation

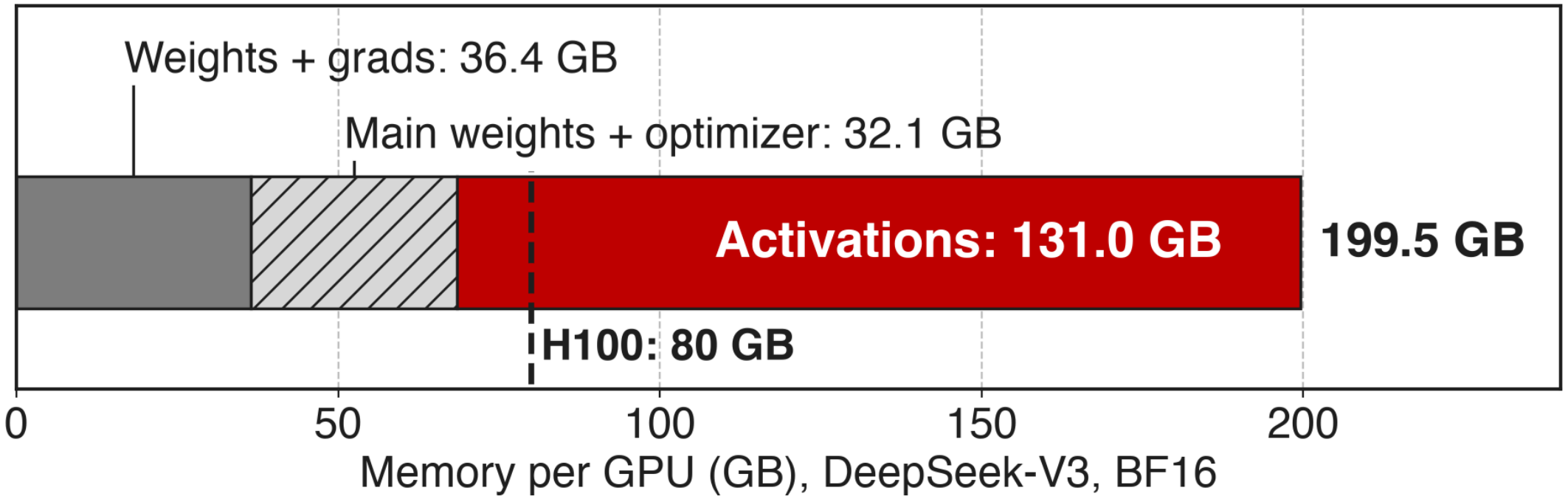
Recompute only the wide, cheap activations of an MoE layer

NVIDIA · Megatron-Core MoE (v0.13+)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Memory & Offload Scheduling · arxiv.org/abs/2603.07685

Activations are two-thirds of MoE memory



DeepSeek-V3 (685B total, 37B active), BF16, PP4 × VPP4 × EP64 on 256 GPUs. 131 of 199.5 GB = 66% (derived)

131 GB of activations alone overflow an 80 GB H100

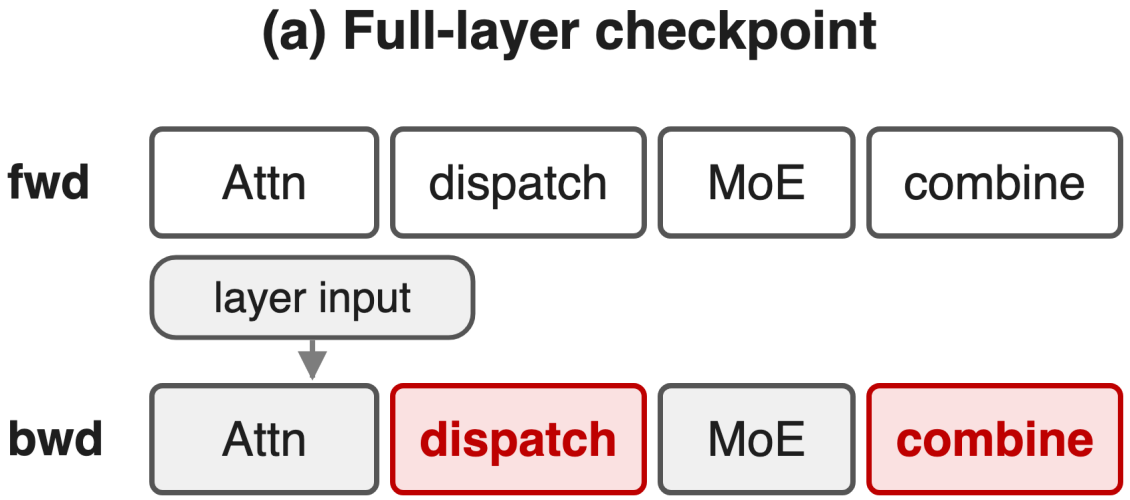
Full recompute costs a third more compute

~33%

extra compute for full-layer recomputation, and in MoE it also replays the EP all-to-all

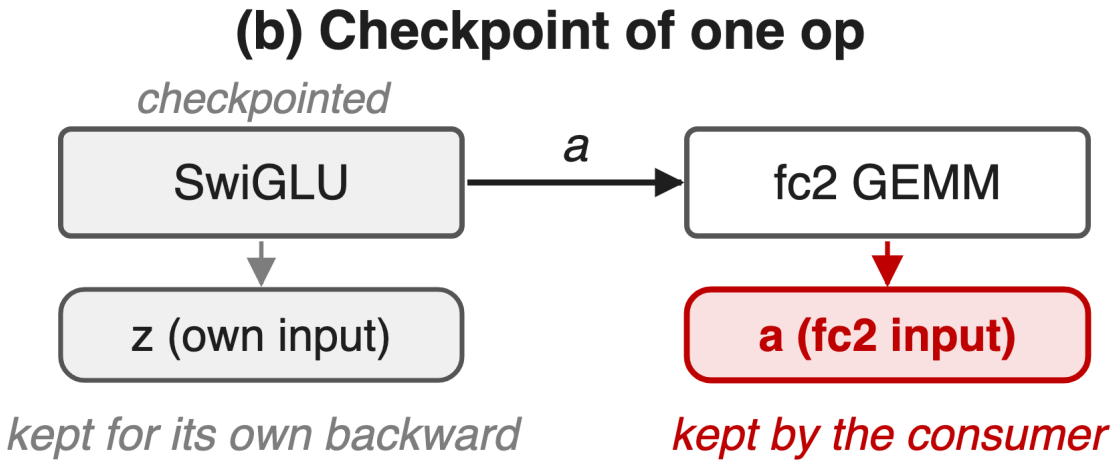
Megatron-Core MoE technical report, arXiv 2603.07685, §4.1.4 (p.22)

Checkpointing is too coarse or frees nothing



whole layer replayed in backward

incl. EP all-to-all



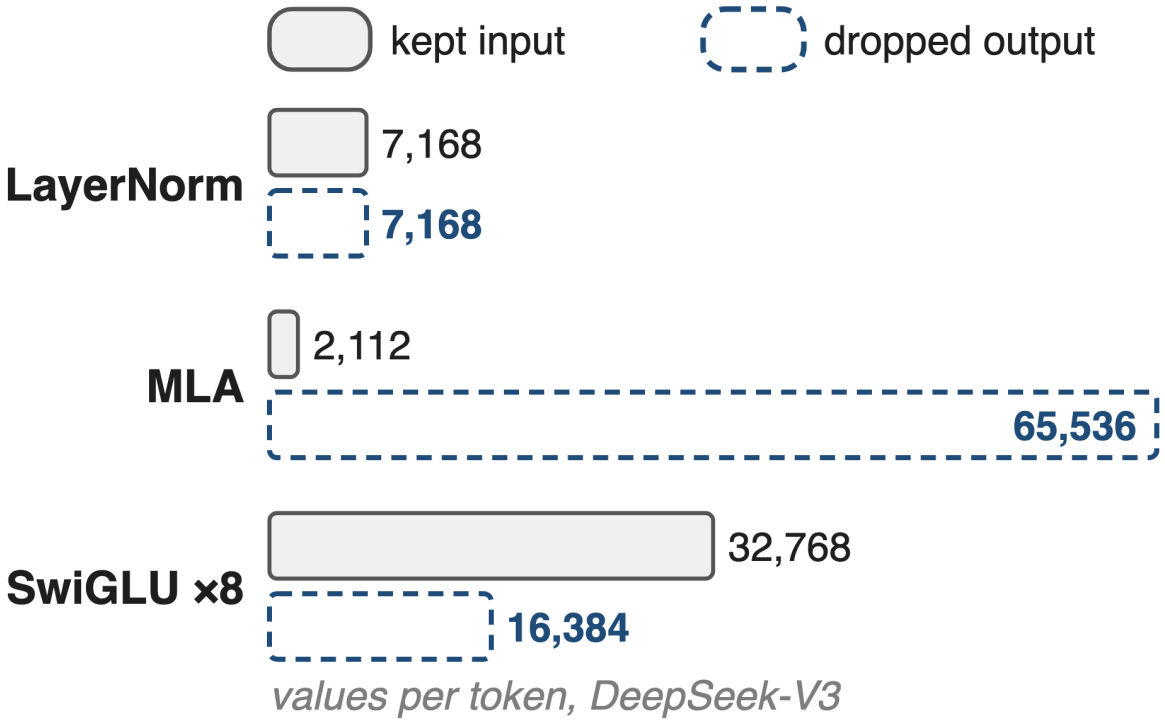
output stays alive anyway

nothing freed

The memory sits in outputs that the next GEMM saves as its input

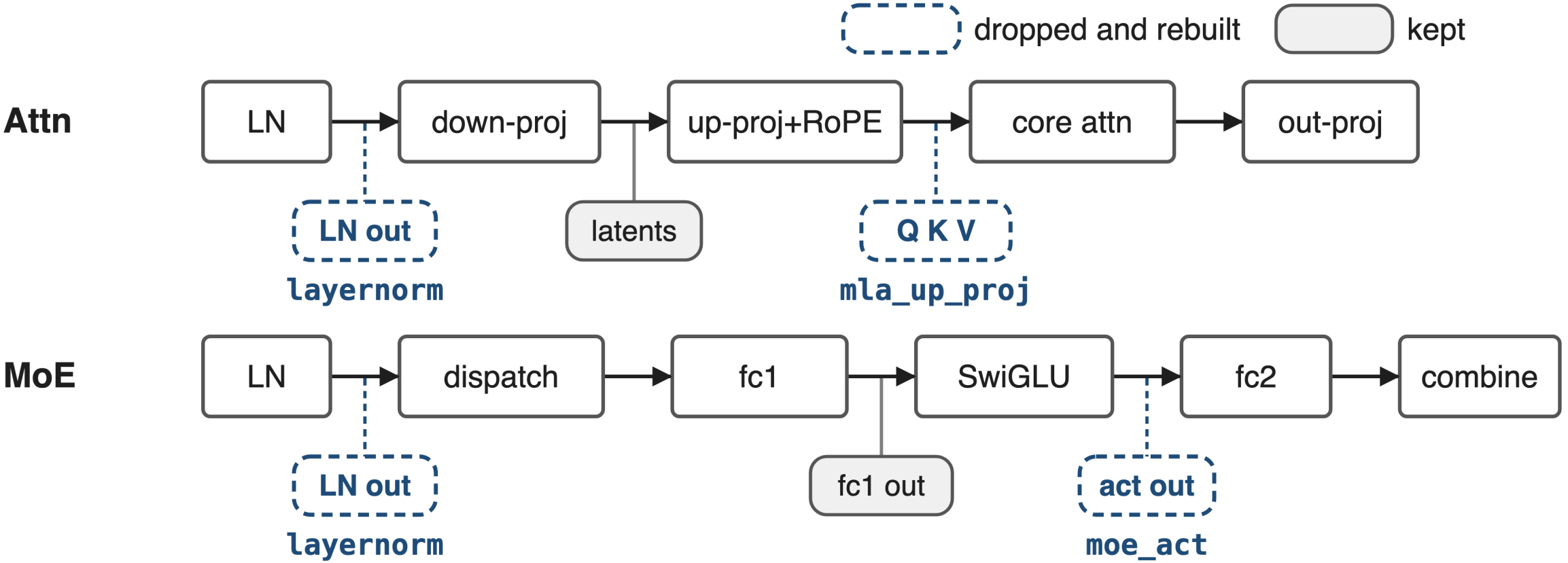
Insight: saved outputs are cheap to re-derive

- Each output is **one cheap op** away from an input kept anyway
- MLA: Q, K, V hold **31×** the values of their latents (analytic)



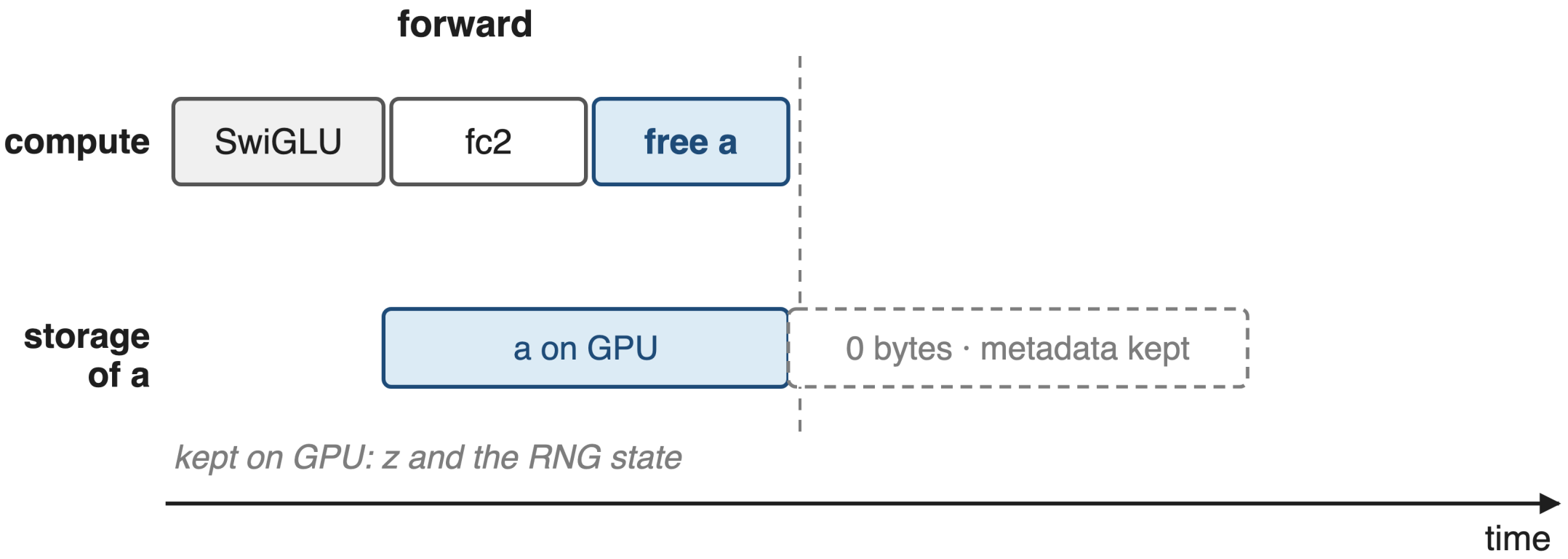
Drop the output, keep the input, rebuild it just before backward

Three modules drop their outputs in every layer

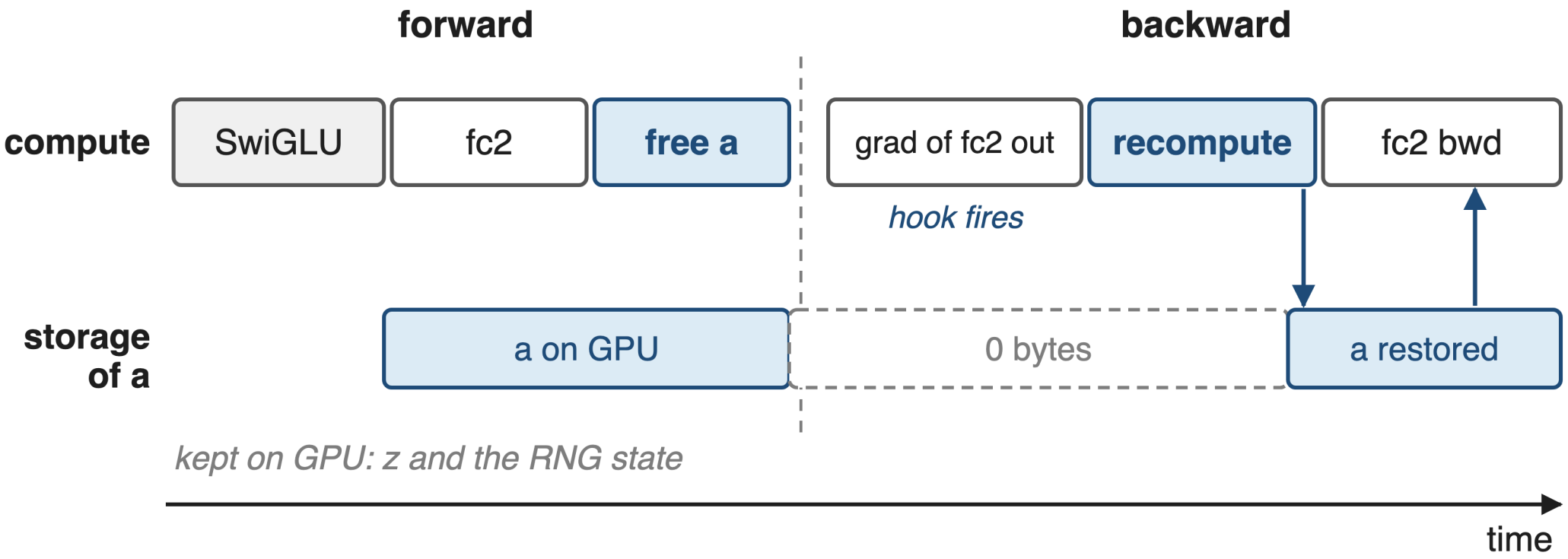


`--recompute-granularity selective --recompute-modules layernorm mla_up_proj moe_act` *(MCore v0.13; FP8 support v0.14)*

❶ Free the output, rebuild it from a grad hook



❶ Free the output, rebuild it from a grad hook



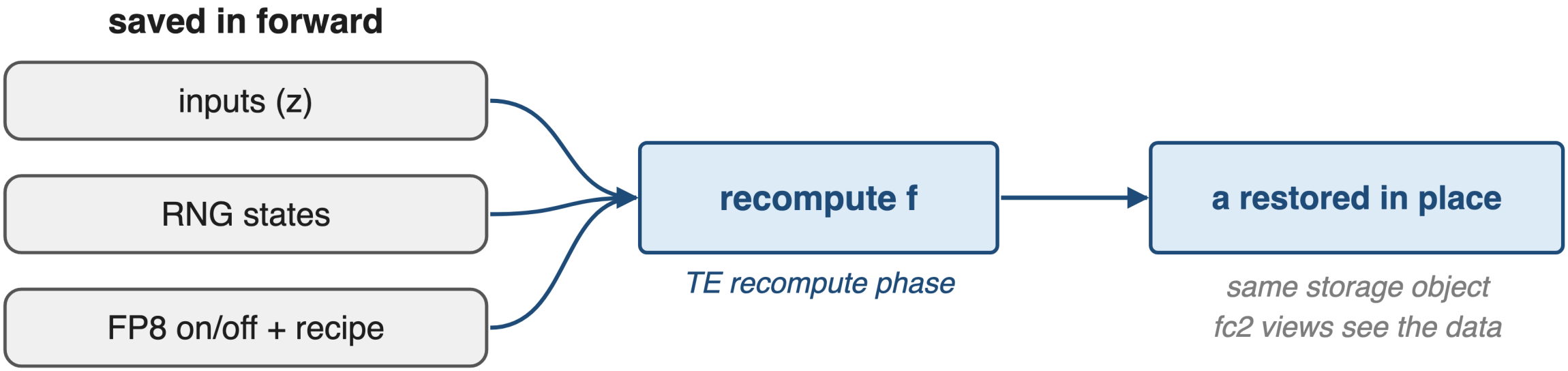
Running example: SwiGLU output a, consumed and saved by expert fc2

❷ Drop what is big to store, cheap to redo

module	drops (output)	keeps (input)	re-runs
`layernorm`	2 norm outputs	norm inputs	1 norm each
`mla_up_proj`	Q K V (128 heads)	MLA latents	2 GEMMs + RoPE
`moe_act`	SwiGLU output	fc1 output	1 elementwise op
`core_attn · mlp`	inner activations	module input	whole submodule
`moe`	inner activations	module input	**experts + all-to-all**

shaded rows: output-discarding checkpoint · others: standard checkpoint

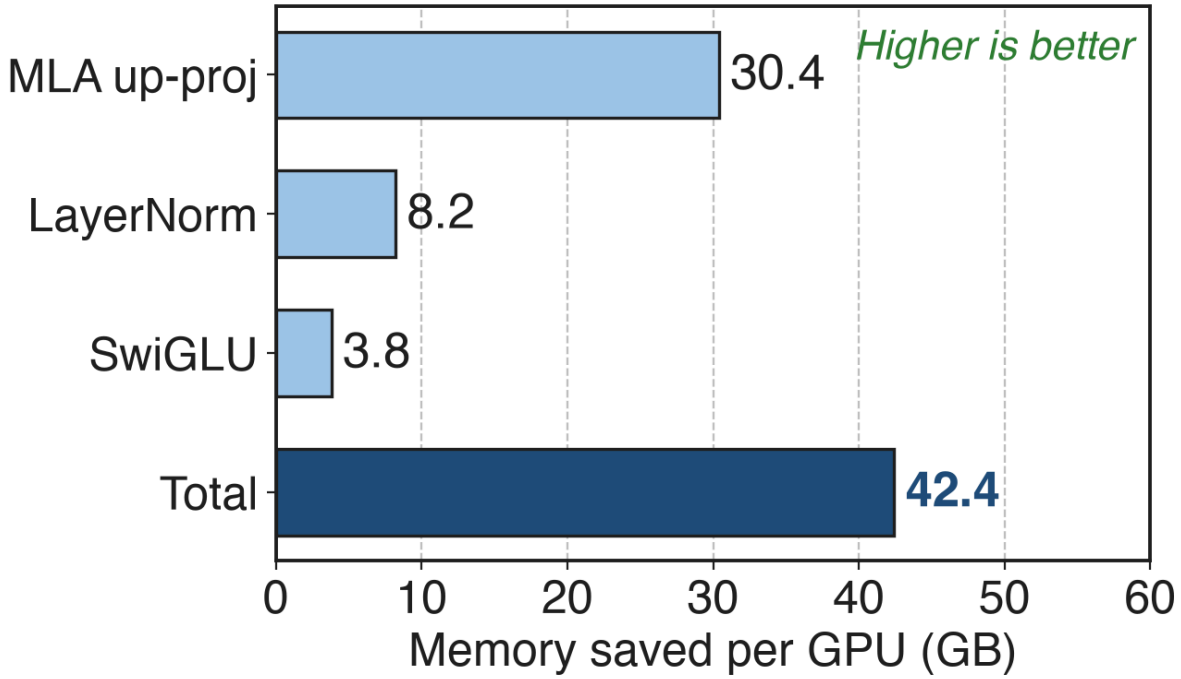
❸ Replay with the forward's RNG and FP8 recipe

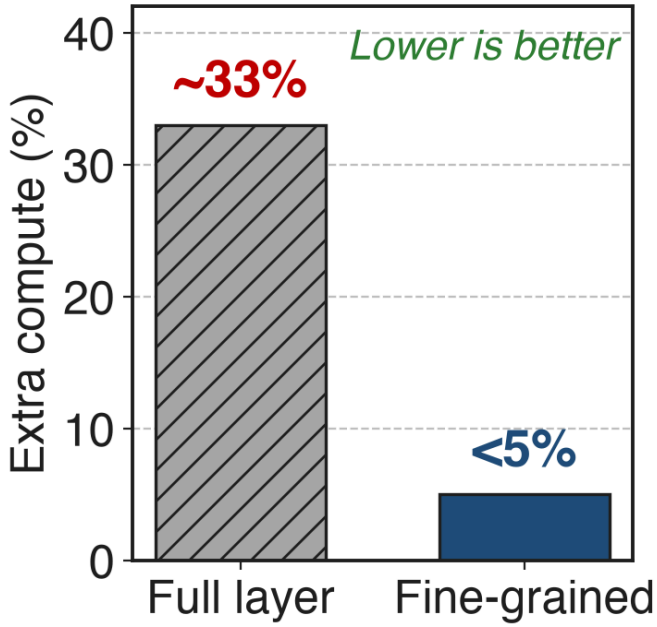


FP8 for fine-grained recompute since MCore v0.14; `moe_act` / `layernorm` under FP8 need TE ≥ 2.6 and no delayed scaling

The rebuilt tensor must match the forward value exactly

42.4 GB saved per GPU for under 5% compute

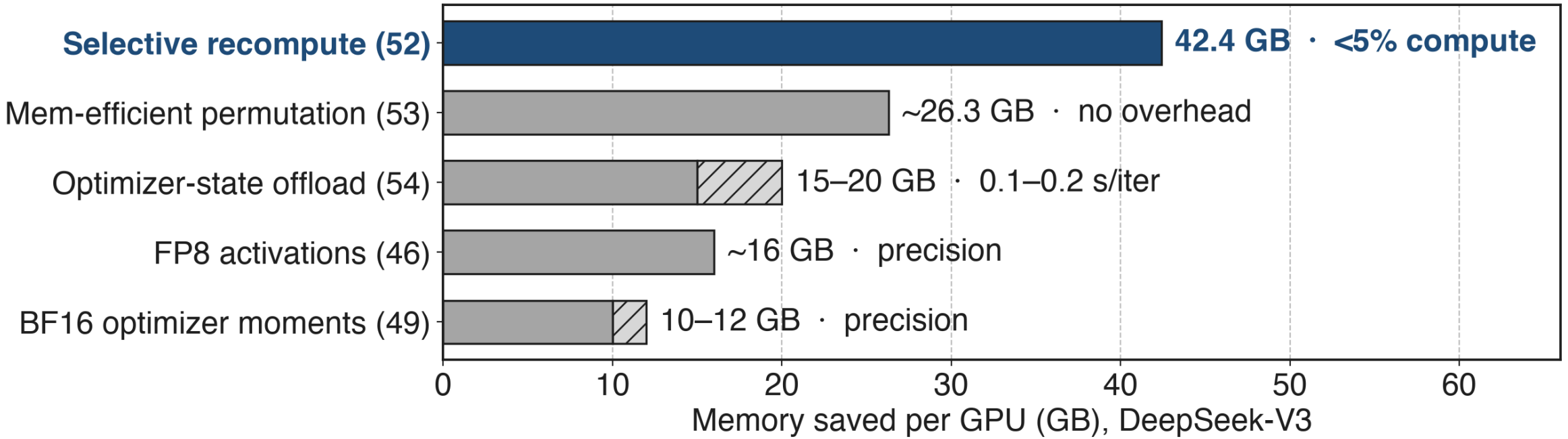




DeepSeek-V3, PP4 × VPP4 × EP64, 256 GPUs (Table 4). Compute: full layer ~33% (§4.1.4); fine-grained "less than 5%" (upper bound)

MLA up-projection alone frees 30.4 GB per GPU

The largest single activation saver in the toolbox



Each saving is reported separately against the 199.5 GB BF16 baseline (Table 3), so they do not simply add up

Together they bring DeepSeek-V3 from 199.5 GB to under 80 GB per GPU

At 64K, recompute everything but attention

- **Core attention** (SDPA)
- **72%** of compute at 64K
- Recompute: **+18%** compute, **–16%** speed, **9 GB** saved

- **Non-attention modules**
- **89.8 GB** saved in total
- Same or lower speed cost

Choose by memory-to-compute ratio; the best set shifts with sequence length

H100 recomputes four modules, GB200 only one

	LayerNorm	MLA up-proj	SwiGLU	dense MLP	MoE down-proj
DeepSeek-V3 · H100	recompute	recompute	recompute	recompute	keep
DeepSeek-V3 · GB200	keep	keep	keep	recompute	keep
Qwen3-235B · GB200/H100	recompute	no MLA	recompute	keep	keep
Kimi K2 · H800	recompute	recompute	recompute	not stated	recompute

DeepSeek-V3: 1,024 H100 at 368 TFLOPS/GPU; 256 GB200 at 1,048 (Table 17). Qwen3-235B: App. B.2. Kimi K2: §2.4.3 of its report

The recompute set is a per-platform knob, not a fixed recipe

The cost: compute, prerequisites, and conflicts

- **Extra compute**: under 5%, not zero
- `moe` replays all-to-all; blocks EP overlap (deck 11)
- Fused attention makes `core_attn` moot
- `moe_act` needs grouped GEMM; `mla_up_proj` needs MLA
- FP8: TE ≥ 2.6, no delayed scaling
- `shared_experts` excludes shared-expert overlap

Summary

- **131 GB** of activations per GPU; full-layer recompute costs **~33%**
- **Insight**: many saved outputs are one cheap op from a kept input
- Drop outputs, keep inputs, rebuild from a grad hook, FP8-exact
- **42.4 GB** saved per GPU for **< 5%** extra compute

https://arxiv.org/abs/2603.07685